

Phase 1 Review Packet

Authentication over Better Auth, rate-limited code issuance, a configurable commission engine with frozen fee snapshots, and a ledger proven under concurrent load. This packet separates what was *executed* from what was merely written, and separates what a machine proved from what I proved on my own workstation.

Branch **claude/create-app-repository-1cxsih**

Head **fcb61db**

2 commits

Tests **146 passed • 3 skipped**

GitHub CI **green (run #4)**

Audit **0 advisories**

1

How to read this

Three words are used with fixed meanings throughout, and nowhere else:

WORD	MEANS EXACTLY
TESTED	An automated test or a command was executed and its result observed. The command that produced it is quoted, so you can run it.
NOT TESTED	The code exists and is reviewable, but no test exercises it. Read it, do not trust it.
NOT IMPLEMENTED	It does not exist. There is no stub pretending otherwise.

ON THE WORD "VERIFIED"

It appears in this document only where a test actually ran. Every green mark below carries a provenance column saying *where* it ran — my workstation, GitHub CI, or both. Section 3 exists solely for that distinction, because you asked for it and because the last packet got it wrong.

A note on the numbering

There are now two documents in `docs/review/` with "Phase 1" in the title, and that is my fault, not a mistake in your reading. The earlier one — `phase-1.html` , sent as `Afrinext-Phase-1-Review-Packet.pdf` — was written when the architecture brief was itself called "Phase 1". The sequence you have since set is **architecture** → **Phase 0** → **Phase 1 (money spine)**. This is the money spine. The earlier file keeps its name and its already-delivered PDF rather than being renamed underneath you; this one is `phase-1-money-spine.html` .

Where the time is worth spending

Suggested split of a ~2 hour review: ~1h20 code · ~40 min sections 14-16

Sections 14 to 16 are the ones that need a decision from you rather than an opinion. In particular, **section 15 contains a security finding I did not fix**, because fixing it properly is a design change and you told me to stop at decisions that materially affect authentication.

What shipped, and against which decision

Every item below traces to a numbered decision in your review. Nothing was built that you did not ask for.

#	YOUR DECISION	WHAT WAS BUILT	STATUS
1	Adopt Better Auth for authentication, without letting it become authorization	Better Auth 1.7.1 over the <code>pg</code> Pool. Its four tables are declared in our schema and created by our migrations. <code>resolveActor()</code> returns an identity and nothing else; permissions still come only from <code>authorize()</code> .	TESTED
2	Do not blindly replace the users/roles/permissions schema	The Phase 0 schema is intact. <code>users</code> gained one nullable column, <code>auth_user_id</code> . Roles, permissions and scoped assignments are untouched.	TESTED
3	One session store, not two	Our <code>sessions</code> and <code>user_identities</code> tables were dropped in migration 0003. Better Auth's <code>session</code> is the only session store. <code>packages/core/src/auth/session.ts</code> was deleted.	TESTED
4	OTP issuance must never remain unlimited	<code>packages/core/ratelimit</code> : fixed-window counters incremented atomically in PostgreSQL. 5 codes per identifier per hour, 20 per IP per hour, 60 s cooldown, 5 verification attempts. A refusal is a 429 with <code>Retry-After</code> .	TESTED
5	Step-up re-verification before payout-sensitive actions	<code>session.elevatedAt</code> plus <code>markElevated()</code> / <code>requireElevation()</code> , 10-minute window. Step-up issuance is limited harder than sign-in: 3 per hour.	BUILT, NOT WIRED TO A PAYOUT
6	Keep script for now; make a later move to Argon2id possible	Better Auth's hasher is replaced with ours. Parameters stay encoded in the hash, so <code>needsRehash()</code> can drive the migration without invalidating a single stored credential.	TESTED
7	Configurable commissions, resolved by rule, frozen per order	<code>commission_rules</code> + <code>fee_schedules</code> + <code>fee_lines</code> . Rules resolve most-specific-first and deterministically. A schedule is frozen at capture, must total the gross, and is append-only in the database.	TESTED
8	Prove the ledger under concurrency, not by argument	Six concurrent tests. They found a real deadlock — see section 10. That is the single most valuable thing in this phase.	TESTED
9	Audit what matters, never the secret	<code>auth.otp.sent</code> , <code>auth.otp.rate_limited</code> , <code>auth.user.provisioned</code> . The code itself is never audited and never logged.	TESTED
10	Do not fake iPayMoney	Unchanged from Phase 0. The adapter still throws <code>ProviderNotConfiguredError</code> . Three	NOT IMPLEMENTED

#	YOUR DECISION	WHAT WAS BUILT	STATUS
		integration tests remain skipped and are counted as skipped everywhere in this packet.	

THE TWO MID-PHASE DECISIONS YOU APPROVED

Better Auth 1.7.1 requires `drizzle-orm ≥ 0.45.2`, above the 0.38.4 the review had confirmed. I stopped and asked rather than upgrading Drizzle on my own initiative. You chose: (a) keep Better Auth's four tables in *our* migration chain rather than letting its adapter own them, and (b) make its `session` the sole session store. Both are implemented exactly as decided. The Drizzle version moved to 0.45.2 later and for a different reason — a published CVE, section 11 — not to satisfy the adapter, which is still not used.

You asked for these to be distinguished, so they are separated here and never merged anywhere else in this document.

What GitHub CI proved

CI runs on a clean Ubuntu runner against a postgres:16 service container. It installs from a frozen lockfile, applies the migrations, **then drops the schema and rebuilds it from zero**, then seeds, then runs everything. Nothing on my workstation is involved.

RUN	COMMIT	JOBS	RESULT	FINISHED (UTC)
#4	fcb61db	both	SUCCESS	2026-08-19 18:26
#3	5cbbac5	both	SUCCESS	2026-08-19 18:08
#2	ee04b8f	both	SUCCESS	2026-08-19 10:42
#1	87743b6	both	SUCCESS	2026-08-19 10:35

Run #3, on the main Phase 1 commit, ran these steps and every one of them passed:

JOB	STEPS THAT CONSTITUTE THE PROOF
lint · typecheck · test	create the test database · migrate · rebuild the schema from scratch · seed · pnpm lint · pnpm typecheck · pnpm test · build the web app · ledger concurrency and idempotency under parallel load · assert the ledger reconciles after the suite
dependency scan	pnpm audit --audit-level high (blocks the build) · full advisory report uploaded as a downloadable artifact

What only my workstation proved

These were executed, and their output is quoted in this packet, but they are **not** in CI. Treat them as one person's observation on one machine, reproducible by you with the commands in section 13.

CLAIM	HOW IT WAS PRODUCED	IN CI?
HTTP status codes on the v1 routes (401 / 403 / 200)	<code>curl</code> and a Playwright request context against <code>next start</code>	NO
A signed-in <i>member</i> is refused <code>/api/v1/admin/ledger</code> with 403	Real browser session, then a request on that session	NO
Rate-limit refusal is a live 429 with <code>Retry-After: 6</code>	<code>curl -D-</code> against the running app	STATUS ASSERTED IN A UNIT TEST; THE HTTP ROUND TRIP IS NOT
The sign-in → code → wallet journey renders	Playwright driving the real UI; screenshots in section 12	NO
Locale routing: <code>/fr</code> and <code>/en</code> 200, <code>/xx</code> 404	<code>curl</code>	NO

THE HONEST READING OF THAT TABLE

There is **no browser-level test in CI**. Every HTTP and UI claim in this packet rests on me having run it once, today, on this commit. If a future change breaks the 403 on the admin route, CI will not notice. Making that impossible is a Phase 2 item and is listed in section 16 for your decision, not assumed.

Two commits on `claude/create-app-repository-1cxsih`:

COMMIT	WHAT IT IS	SCALE
5cbbac5	Phase 1: Better Auth, OTP limits, audit, concurrency, commission engine	54 files · +10 993 / -1 078
fc61db	Return 429, not 500, when OTP issuance is refused	2 files · small

`pnpm-lock.yaml` accounts for about 1 755 of those changed lines. The reviewable surface is roughly 2 300 lines of hand-written code and SQL, listed below.

New — read these

FILE	LINES	WHY IT EXISTS
<code>packages/core/src/commissions/index.ts</code>	198	Rule resolution and the frozen fee snapshot. The most consequential new file.
<code>packages/core/src/auth/better-auth.ts</code>	195	The whole Better Auth configuration, in one reviewable place.
<code>packages/core/src/auth/session-bridge.ts</code>	140	Session → identity. Deliberately returns no permissions.
<code>packages/core/src/ratelimit/index.ts</code>	136	Atomic counters and the OTP policy constants.
<code>packages/db/src/schema/commerce.ts</code>	107	Commission rules, fee schedules, fee lines, settlement holds.
<code>packages/db/src/schema/better-auth.ts</code>	90	Better Auth's four tables, declared by us so they are diffed and rebuilt like everything else.
<code>packages/core/src/auth/messaging.ts</code>	74	The SMS/email sender seam. <code>ConsoleSender</code> refuses to run in production without an explicit opt-in.
<code>packages/core/src/auth/errors.ts</code>	23	<code>RateLimitedError</code> , <code>ElevationRequiredError</code> .

Migrations

MIGRATION	LINES	EFFECT
0002_better_auth_tables	83	Creates <code>user</code> , <code>session</code> , <code>account</code> , <code>verification</code> .
0003_retire_superseded_auth_tables	13	Drops our <code>sessions</code> and <code>user_identities</code> . Two sources of truth become one.
0004_commission_and_settlement	66	Commission rules, fee schedules, fee lines, settlement holds.
0005_fee_snapshot_immutable	43	Append-only triggers on the fee tables, and the deferred constraint that fee lines must total the gross.

New in the web app

Route handlers `/api/auth/...all` , `/api/v1/me` , `/api/v1/wallet` , `/api/v1/admin/ledger` ; pages `/[locale]` , `/[locale]/sign-in` , `/[locale]/wallet` ; and `src/lib/{auth,session,api,auth-client}.ts` . All of `apps/web/src` is 2 805 lines including the untouched Phase 0 prototype.

Deleted

`packages/core/src/auth/session.ts` and its tests. Superseded by Better Auth's session; leaving it would have been the second source of truth you rejected.

Current state of the schema, read out of a live PostgreSQL 16 after applying all six migrations:

OBJECT	COUNT	DETAIL
Tables	28	24 ours, 4 Better Auth's — all declared in our schema.
Views	1	<code>ledger_account_balances_derived</code> — the truth the cache is checked against.
Migrations	6	All applied; all replayed from zero in CI.
Triggers enforcing invariants	9	Listed below.
Named check constraints	18	Including <code>otp_purpose_step_up_only</code> , new this phase.

The nine triggers

TRIGGER	ON	WHAT IT MAKES IMPOSSIBLE
<code>ledger_entries_must_balance</code>	<code>ledger_entries</code>	An unbalanced transaction. Deferred, so it fires at commit.
<code>ledger_entries_currency_matches_account</code>	<code>ledger_entries</code>	Posting XOF into an EUR account.
<code>ledger_entries_append_only</code>	<code>ledger_entries</code>	UPDATE or DELETE on a posted entry.
<code>ledger_transactions_append_only</code>	<code>ledger_transactions</code>	Rewriting a transaction header.
<code>fee_lines_must_total_gross</code>	<code>fee_lines</code>	A fee snapshot whose lines do not add up to the gross. New this phase.
<code>fee_lines_append_only</code>	<code>fee_lines</code>	Restating a past order's commission. New this phase.
<code>fee_schedules_append_only</code>	<code>fee_schedules</code>	Editing a frozen schedule. New this phase.
<code>audit_logs_append_only</code>	<code>audit_logs</code>	Erasing your own tracks.
<code>consent_records_append_only</code>	<code>consent_records</code>	Rewriting what someone agreed to.

WHY THE FEE TRIGGERS MATTER MORE THAN THEY LOOK

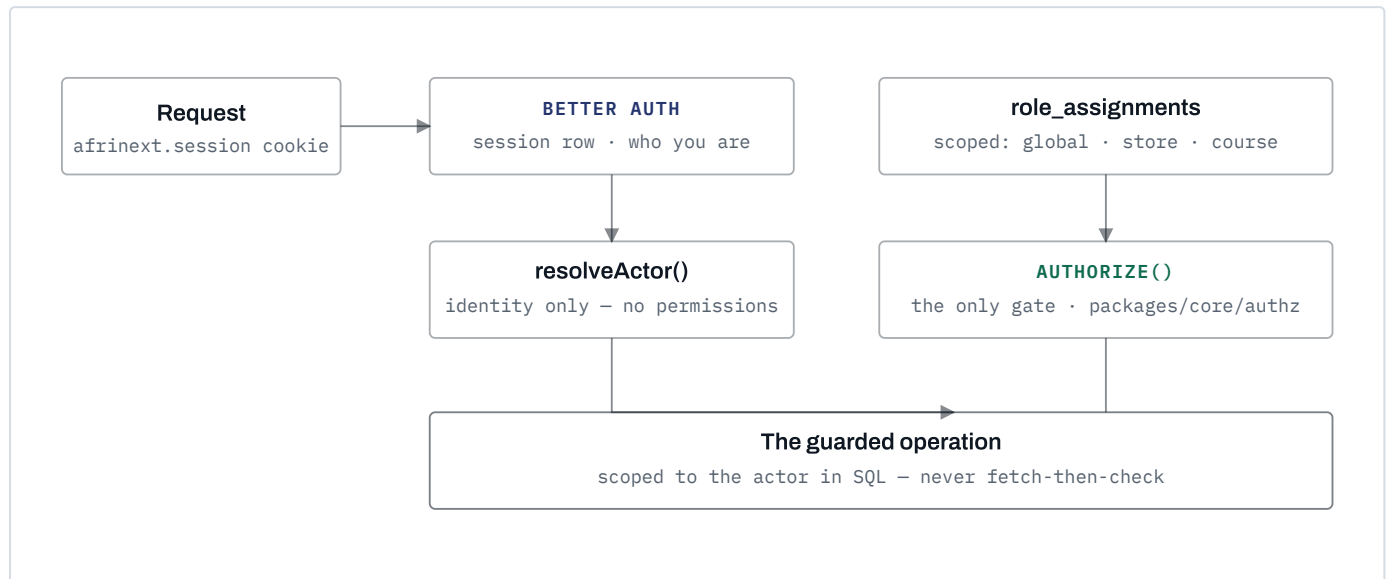
A commission rule is *configuration* — it will change. If a past order's fee lines could be recomputed from the current rule, then changing a rule would silently restate history and the ledger would disagree with the order. Freezing the snapshot in the database, rather than in application code, means a future service written in another language cannot get this wrong either.

Tables dropped

`sessions` and `user_identities`. These were Phase 0 work that your review superseded. They are dropped, not left orphaned — an unused table with credentials in it is a liability, not a spare.

Authentication — and the line it does not cross

Your instruction was that authentication must not become authorization. That is a structural claim, so here is the structure.



Better Auth answers exactly one question: **who is this?** It has no path to the right-hand column. A session that Better Auth considers perfectly valid still yields nothing but a user id, and every capability is a separate lookup against scoped role assignments.

What that buys, concretely

- A suspended user gets **no actor at all**, even holding a valid session cookie — the suspension is checked where identity is resolved, not where each feature remembers to check.
- The signed-in member in section 12 gets 403 `authz.denied` on the admin ledger route. Authentication succeeded completely; it simply grants nothing.
- Replacing Better Auth later touches `better-auth.ts` and `session-bridge.ts`. It does not touch a single permission check.

The four tables, and why we own them

Better Auth ships a Drizzle adapter that would create and own `user`, `session`, `account` and `verification`. We do not use it. The tables are declared in `packages/db/src/schema/better-auth.ts` and created by migration 0002, which means they are reviewed in a pull request, rebuilt from zero in CI, and diffable like everything else.

The risk that creates is drift: a Better Auth upgrade could expect a column we do not have. `auth/schema-drift.test.ts` exists for exactly that — it compares what Better Auth's own migration planner wants against what we declare, and fails the build on a mismatch. **TESTED** — 4 tests.

Sessions are rows, so revocation is one statement

`revokeSession()` and `revokeAllSessionsForUser()` delete rows. There is no token to wait out. Both are tested, including that a revoked session no longer resolves.

OTP issuance is limited, not just OTP attempts

Bounding verification attempts alone protects nothing: someone who can request unlimited fresh codes has unlimited attempts, and in Niger every SMS is a real cost. So issuance is limited too.

RULE	LIMIT	WINDOW	BUCKET
Codes per identifier	5	1 hour	otp:send:id:<phone>
Cooldown between codes	1	60 seconds	otp:cooldown:id:<phone>
Codes per IP	20	1 hour	otp:send:ip:<ip>
Step-up challenges per user	3	1 hour	stepup:send:user:<id>
Verification attempts per code	5	—	enforced by Better Auth

The counter is one atomic statement, so two concurrent requests cannot both read "4 used" and both proceed:

```
insert into rate_limit_counters (bucket, window_start, count)
values ($1, $2, 1)
on conflict (bucket, window_start)
do update set count = rate_limit_counters.count + 1
returning count
```

A request that exceeds the limit is *still counted*. That is deliberate: hammering a blocked bucket must not let it drain early. **TESTED** — 14 tests, including a concurrent one that pins the clock so it cannot race the window boundary.

A refusal is a 429, not a 500 **FIXED IN FCB61DB**

The rate limiter was working from the first commit — issuance was refused, both outcomes audited. But an unrecognised throw out of a Better Auth callback becomes a bare 500 with an empty body, so the caller could not distinguish a refusal from a server fault, and no client could implement a sensible retry. It now leaves the handler as a real 429:

```
$ curl -D- -X POST localhost:3000/api/auth/phone-number/send-otp \
-H 'content-type: application/json' -d '{"phoneNumber":"+22790123456"}'

- first request -
HTTP/1.1 200 OK
{"message":"code sent"}

- immediately again -
HTTP/1.1 429 TOO_MANY_REQUESTS
retry-after: 6
{"code":"ratelimit.exceeded","message":"Too many verification code requests. Try again in 6 second(s"}
```

The test now asserts `statusCode === 429` and the error code, rather than merely that something threw — which is what would have caught it the first time.

The secret is never written down

`auth.otp.sent` records the channel and which sender was used. `auth.otp.rate_limited` records the count and the limit. Neither records the code, and the structured logger redacts by key name recursively, so a code cannot reach a log through a nested object either.

Two guards that refuse to run in production

- `ConsoleSender` prints codes to the console. It **throws on startup** in production unless `ALLOW_CONSOLE_SENDER=yes` is set explicitly — the same pattern as the destructive-reset guard.
- The mock payment provider still refuses to load in production, unchanged from Phase 0.

Passwords

Better Auth's default hasher is replaced with Afrinext's scrypt, per your decision to keep scrypt for now. Parameters remain encoded inside each hash, so a later move to Argon2id can verify old hashes while writing new ones and no stored credential is invalidated.

Commissions are configuration, resolved by rule

No rate is a constant anywhere in the code. A rule may name a country, a category, a store, or none of them, and carries a rate in basis points, a fixed minor-unit amount, or both. Resolution is **most specific first**:

```
order by specificity desc, -- a store rule beats a country rule beats the global default
       priority desc,     -- explicit tie-break you control
       effective_from desc,
       id desc             -- total order: the same inputs always give the same answer
limit 1
```

That last line matters more than it looks. Without a total order, two equally-specific rules would resolve by whatever order the planner felt like, and a settlement could not be re-derived years later. **TESTED** — 13 tests, including specificity precedence, tie-breaking, and time windows.

If no rule is in force, `freezeSchedule` **throws** rather than defaulting to zero or to some hard-coded rate. A sale that cannot say what the platform charged does not settle.

Seeded launch rules, all in basis points of gross, all country-agnostic so Niger is data and not a special case:

TRANSACTION TYPE	RATE	BPS
course	18.00 %	1800
digital	18.00 %	1800
service	15.00 %	1500
physical	12.00 %	1200

Referral comes out of the platform's cut, never off the top

A referral line is declared with `basis: { line: "platform" }` and `deductFrom: "platform"`. The seller's share is therefore untouched by whether a referral existed. This is the rule that is easiest to get quietly wrong and hardest to detect afterwards, so it is expressed in the data structure rather than in arithmetic.

The residual rule

Every non-seller line rounds half-up; the seller receives `gross -  $\Sigma$ (other lines)`. The seller therefore absorbs the rounding remainder and the split always re-sums to the gross exactly. In XOF this is not academic — **XOF has zero decimals**, so the minor unit *is* the franc and a one-unit rounding error is a franc that either exists or does not.

The snapshot is frozen in the database

Once `freezeSchedule` commits, the outcome lives in `fee_lines` and the database refuses to change it. A deferred constraint trigger additionally asserts the lines total the gross, so a split that does not add up cannot be written at all — not by this code, not by a future service, not by someone with `psql`.

The reconciliation gate, run on this commit

A real capture → settlement → payout cycle is posted, then the cached balances are compared against `sum(direction × amount_minor)` derived from the entries:

```
{
  "scenario": { "gross": "10000", "seller": "8200",
               "platform": "1620", "referral": "180" },
  "checkedAccounts": 8,
  "drift": 0,
  "unbalancedTransactions": 0,
  "netByCurrency": { "XOF": "0" }
}
Ledger reconciliation clean over a real capture → settlement → payout cycle.
```

Read the numbers: gross 10 000 XOF at 18 % gives a platform cut of 1 800, of which a 10 % referral takes 180, leaving the platform 1 620 — and the seller 8 200, which is unchanged by the referral existing.

TESTED in CI as the final step of every run, so a change that breaks reconciliation cannot merge quietly.

What is deliberately absent

No screen, field, API response or database column calls a seller balance a deposit or protected funds. `/api/v1/wallet` returns, in the payload itself:

```
"note": "Balances are contractual entitlements against Afrinext, not deposits."
```

The ledger records entitlement, not custody — and per your instruction, company registration does not establish authorization for regulated payment activity. Nothing in this phase claims otherwise.

146 passed, 3 skipped, across 15 files. Executed on this commit, both locally and on GitHub CI. The three skipped are the iPayMoney integration tests and are skipped because there is nothing to integrate against — they are counted as skipped everywhere in this packet and never folded into the passing number.

FILE	TESTS	WHAT IT ACTUALLY PROVES
auth/auth.test.ts	19	Password hashing, rehash detection, OTP challenge lifecycle, attempt bounds.
ratelimit/ratelimit.test.ts	14	Windows, cooldowns, per-IP rules, and that concurrent consumers cannot overshoot a limit.
ledger/ledger.test.ts	14	Double-entry invariants, append-only enforcement, currency matching, idempotency.
commissions/commissions.test.ts	13	Rule specificity, deterministic tie-breaks, time windows, frozen snapshots, refusal when no rule applies.
money/money.test.ts	13	Minor units, zero-decimal currencies, refusal of floats. Property-based.
payments/payments.test.ts	13	The provider abstraction; the mock refusing to load in production.
auth/flow.test.ts	11	Phone sign-in end to end, domain provisioning, revocation, elevation, the 429.
authz/authorize.test.ts	11	Scoped permissions, denial by default, suspended users.
money/allocate.test.ts	9	The residual rule. Property-based: splits always re-sum to the gross.
money/currency.test.ts	7	Exponents read from the table, not assumed to be 2.
ledger/concurrency.test.ts	6	Parallel posting, lost-update resistance, idempotency under races. Found the deadlock.
consent/consent.test.ts	6	Versioned documents, append-only consent records.
observability/logger.test.ts	5	Recursive secret redaction by key name.
auth/schema-drift.test.ts	4	Our declared Better Auth tables still match what Better Auth expects.
payments/ipaymoney.integration.test.ts	4	1 asserts the adapter refuses to pretend. 3 skipped — no provider to test against.

Test code is **2 181 lines** against **4 201 lines** of package source — a ratio of about 0.52, concentrated on money and authorization rather than spread evenly.

THIS IS THE FINDING OF THE PHASE

You required concurrency tests. They immediately found a production deadlock in ledger posting that no amount of reading the code had surfaced — including my own reading of it, twice.

What was wrong

`postTransaction` updated the cached balance rows in whatever order the transaction's entries happened to be written. So a transfer $A \rightarrow B$ locked A then B, while a concurrent transfer $B \rightarrow A$ locked B then A. Each held what the other wanted; PostgreSQL detected the cycle and aborted one with a deadlock error.

Under a real marketplace this is not exotic. Two buyers paying two sellers who also buy from each other is enough. It would have appeared as intermittent, unreproducible payment failures in production.

The fix

```
// Lock the balance rows in a deterministic order.
//
// Without this, a transfer A->B and a concurrent transfer B->A each lock
// their two rows in the order the transfers were written, so one holds A and
// wants B while the other holds B and wants A: PostgreSQL detects the cycle
// and aborts one with a deadlock. Sorting by account id means every
// transaction in the system acquires these locks in the same order, which
// makes such a cycle impossible. Found by the concurrent-posting test, not
// by reasoning.
const orderedDelta = [...delta].sort(([a], [b]) => (a < b ? -1 : a > b ? 1 : 0));

for (const [accountId, amount] of orderedDelta) { /* ... */ }
```

Every transaction in the system now acquires balance locks in the same global order, which makes a cycle impossible rather than merely unlikely.

The measurement

CONCURRENCY SUITE	BEFORE	AFTER
Wall time	23 s	0.8 s
Outcome	deadlock aborts, retries	clean

The 23 seconds were the database detecting cycles and the suite retrying. Speed was the symptom; the deadlock was the disease.

This rule is now written into `AGENTS.md` as non-negotiable: anything that locks more than one row sorts first. **TESTED** — the concurrency suite runs as its own named step in CI, so a regression is attributable rather than buried in a 149-test summary.

11 Dependency scanning

Current state, executed on this commit both locally and in CI:

```
$ pnpm audit --audit-level high
No known vulnerabilities found
```

That was not the state when the phase started. The scan found real advisories and they were fixed rather than noted:

PACKAGE	SEVERITY	ISSUE	RESOLUTION
drizzle-orm	HIGH	CVE-2026-39356 — SQL injection through identifiers	Upgraded to 0.45.2. Drizzle stays, per your decision; only the version moved.
vitest	CRITICAL	Advisory in the pinned range	Upgraded to 3.2.7.
vite / esbuild	TRANSITIVE	Pulled in below the patched versions	Pinned by root overrides : vite >=6.4.3 , esbuild >=0.25.0 .

WORTH NAMING PLAINLY

The Drizzle advisory is a SQL-injection class issue in the ORM this project routes *every* financial statement through. It was found by a scan you insisted on, not by review. The dependency-scan job now blocks the build on high and critical, and uploads the full advisory report as a downloadable artifact on every run so a low-severity trend is visible before it becomes a high one.

One piece of dead configuration was also removed this phase: a `peerDependencyRules` override pinning the Drizzle adapter's peer to 0.38.4, which no longer applies now that the adapter is unused and Drizzle is 0.45.2. Leaving it would have masked a future peer mismatch. The lockfile is unchanged by its removal, and `pnpm install --frozen-lockfile` still succeeds.

All of this was produced against a production build (`next build` then `next start`) on this commit. None of it is in CI — see section 3.

HTTP behaviour

ROUTE	ANONYMOUS	SIGNED-IN MEMBER	READING
/api/v1/me	401	200	Returns the identity and its permission list.
/api/v1/wallet	401	200	Entitlements derived from ledger entries, not from the cache.
/api/v1/admin/ledger	401	403	The one that matters. Authentication fully succeeded; it grants nothing.
/fr/wallet	307	200	Redirects to sign-in when signed out.
/fr/sign-in /en/sign-in	200	200	Both locales route.
/xx/sign-in	404	404	An unknown locale is refused, not silently defaulted.

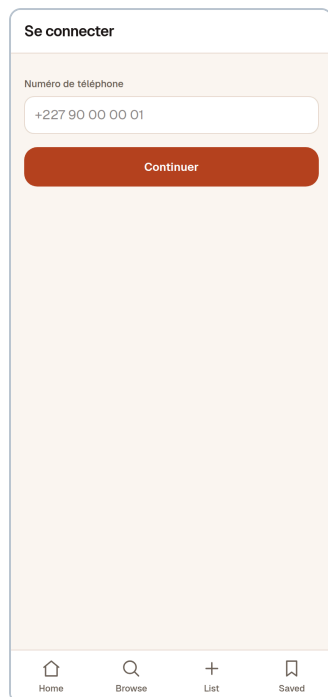
```
- anonymous -
{"error":{"code":"auth.required","message":"Authentication required."}}

- signed-in member, admin route -
{"error":{"code":"authz.denied",
  "message":"Permission "ledger.read" is not granted on global."}}
```

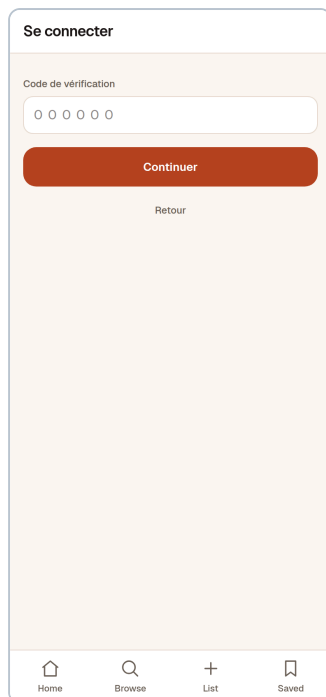
The message names the permission and the scope. That is deliberate: a denial that cannot be diagnosed becomes a denial someone works around.

The journey, end to end

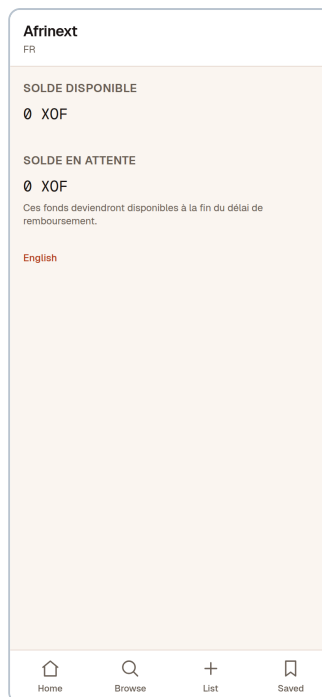
Driven by a real browser at 390×844 — the phone this is built for, not a desktop window narrowed down.



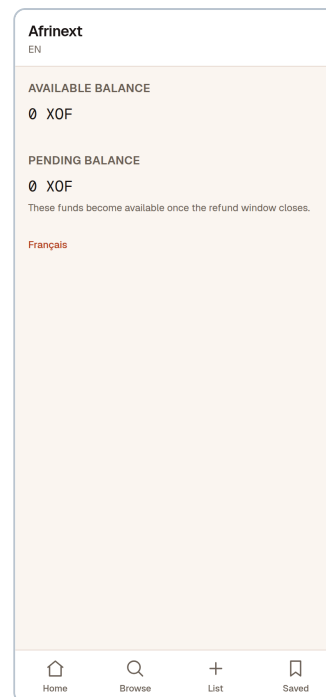
Sign in — The phone step. One field, a tel keyboard, French by default because Niger's default locale is data in the `locales` table, not a constant.



Verification code — The six-digit step. The code was delivered through the sender seam; it is never logged, never audited, and never rendered.



Wallet • FR — 0 XOF, not **0,00 XOF** — the exponent came from the `currencies` table. The pending line reads *funds become available at the end of the refund window*; nothing on this screen calls it a deposit.



Wallet • EN — The same route under `/en`. Locale is a path segment, so both are linkable and an unknown locale 404s rather than silently falling back.

13 Reproduce every claim in this packet

Nothing here needs my machine. Node 22+, pnpm 10+, PostgreSQL 16+.

1 Get the branch and install

```
git clone https://github.com/abdoulasani/Afrinext
cd Afrinext
git checkout claude/create-app-repository-1cxsih
git log --oneline -2          # should show fcb61db, 5cbbac5
pnpm install --frozen-lockfile
```

2 Point it at two databases

```
cp .env.example .env
# set DATABASE_URL and TEST_DATABASE_URL — they must differ.
# The suite truncates every table it touches and refuses to run otherwise;
# that guard fired on me during this phase, working exactly as intended.
```

3 Build the schema and seed it

```
pnpm db:migrate
pnpm --filter @afrinext/core seed
```

4 Run everything

```
pnpm lint          # exits 0
pnpm typecheck     # exits 0
pnpm test          # 146 passed | 3 skipped (149), 15 files
pnpm build         # exits 0
pnpm audit --audit-level high # No known vulnerabilities found
```

5 Prove the ledger separately

```
pnpm --filter @afrinext/core exec vitest run src/ledger/concurrency.test.ts
DATABASE_URL="$TEST_DATABASE_URL" \
  pnpm --filter @afrinext/core exec tsx scripts/reconcile-check.ts
```

The second prints the JSON in section 8. `drift` must be 0 and every currency must net to 0.

6 See the deadlock for yourself

Revert one line — remove the `.sort(...)` in `packages/core/src/ledger/post.ts` — and re-run the concurrency test. It goes

from under a second to roughly twenty-three, with deadlock aborts in the output. Put it back afterwards.

7

Check the HTTP claims

```
ALLOW_CONSOLE_SENDER=yes pnpm start

curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/api/v1/me           # 401
curl -s -o /dev/null -w '%{http_code}\n' localhost:3000/xx/sign-in         # 404

# twice in a row – the second is the rate limiter:
curl -D- -X POST localhost:3000/api/auth/phone-number/send-otp \
  -H 'content-type: application/json' -d '{"phoneNumber":"+22790123456"}'
# 200, then 429 with retry-after
```

For the 403, sign in through the UI at `/fr/sign-in` (the code is printed by `ConsoleSender`), then request `/api/v1/admin/ledger` on that session.

8

Read the CI runs rather than trusting this document

github.com/abdoulasani/Afrinext/actions — runs #1 to #4, all green. Run #4 is this head commit. Every step name is visible; the dependency-scan job attaches its full advisory report as a downloadable artifact.

Read this section as the counterweight to section 9. Everything below is either untested code or absent code, stated plainly.

Built, but not tested

THING	STATUS	WHAT THAT MEANS FOR YOU
Step-up elevation guarding an actual payout	NOT TESTED	The mechanism has tests; nothing yet <i>calls</i> it before moving money, because no payout endpoint exists. The guarantee is unproven in the only context that matters.
Every HTTP route handler	NOT TESTED	No automated test hits a route. The status codes in section 12 came from me running <code>curl</code> once.
Every React component	NOT TESTED	<code>apps/web</code> has no test suite at all. <code>pnpm test</code> in that package prints a placeholder and exits 0.
Email OTP delivery	NOT TESTED	Configured, and phone is the launch path. No email is actually sent by anything.
The consent flow in the UI	NOT TESTED	The tables and core functions are tested. Nothing in the interface presents a document or records agreement yet.
Migration <code>0005</code> has no drizzle-kit snapshot	COSMETIC	It contains only triggers and functions, which drizzle-kit does not model — the same pattern as <code>0001</code> . It applies and replays correctly in CI; it simply will not appear in a generated diff.

Not built

THING	STATUS	NOTE
iPayMoney integration	NOT IMPLEMENTED	The adapter throws <code>ProviderNotConfiguredError</code> . No invented endpoints, no invented auth parameters, no invented webhook payloads, no fabricated success responses. <code>docs/providers/ipaymoney/README.md</code> lists the 13 documentation items needed to build it.
Payouts and withdrawals	NOT IMPLEMENTED	The four-eyes rule — nobody approves their own withdrawal — is designed and has a place in the schema. There is no endpoint to enforce it on yet.
Products, orders, checkout, courses	NOT IMPLEMENTED	Phase 2 and beyond. The commission engine takes a <code>subjectType</code> / <code>subjectId</code> so it is ready for them.
SMS delivery to a real handset	NOT IMPLEMENTED	Only <code>ConsoleSender</code> exists. No SMS provider is chosen, contracted or configured, and none is pretended.
Browser tests in CI	NOT IMPLEMENTED	The gap named in section 3.

RISK 1 – VERIFICATION CODES ARE STORED IN PLAINTEXT

Better Auth writes the OTP into `verification.value` as `<code>:<attempts>`, in the clear. Anyone with read access to that table — a backup, a replica, a leaked dump, a support tool, an SQL-injection foothold — can sign in as any user with a code in flight. Codes live five minutes, which bounds the window but does not close it.

This is **inconsistent with our own standard**: the `otp_challenges` table we wrote in Phase 0 stores `code_hash`, never the code.

I found this while re-verifying the sign-in journey for this packet and **did not fix it**, because the fix is a design change to authentication and you told me to stop at those. What I established:

- Better Auth's `emailOTP` plugin accepts `storeOTP: "hashed"`, defaulting to `"plain"`. One line.
- Its `phoneNumber` plugin — **our launch path** — has no such option. I read the plugin source: it writes `value: `${code}:0`` unconditionally, with no hook to intercept.

So the options are: (a) hash the email path and accept plaintext on the phone path, which is close to cosmetic in a phone-first market; (b) take the phone OTP flow back into our own `otp_challenges` table, which already hashes, and use Better Auth only to mint the session once verification succeeds; (c) accept the risk for launch, documented, with the table's access tightly restricted. **This needs your decision, not mine.**

RISK 2 – NO BROWSER TEST IN CI

The 403 on the admin route is the single most security-relevant behaviour in the running application, and its only proof is that I ran it by hand today. A refactor that broke it would ship green.

RISK 3 – A REFUSED CODE REQUEST STILL WRITES A CODE ROW

Better Auth creates the `verification` row *before* calling our sender, so a rate-limited request has already written a code that will never be delivered. Nothing is sent and no cost is incurred — the limiter does its job — but the row exists and counts toward table growth. Combined with risk 1, an attacker who can read the table does not even need delivery to succeed. Pruning is not implemented.

RISK 4 – THE RATE LIMITER IS PER-INSTANCE-AGNOSTIC BUT NOT PER-INSTANCE-TESTED

Counters live in PostgreSQL and increment atomically, so limits hold across application instances by construction. That property is tested with concurrent callers against one database; it has never run against two application processes, because there has never been more than one.

RISK 5 – REGULATORY POSTURE IS UNCHANGED AND UNRESOLVED

Repeating what the blueprint says, because it does not become less true with more code: NIF and RCCM establish company registration. They do not establish authorization for regulated payment activity, and consent does not substitute for authorization. Nothing in Phase 1 changes this, and no screen implies otherwise. It remains an open item outside engineering's control.

RISK 6 – ONE UN-ADOPTED UPSTREAM DEPENDENCY DECISION

Better Auth is pinned at exactly `1.7.1`, not a range. That is deliberate — an auth library that silently minor-bumps is not something a money system should carry — but it means upgrades are a manual, reviewed act. `schema-drift.test.ts` will fail loudly if an upgrade expects a column we do not declare, which is the point of it.

These are the items that need a yes, a no, or a different instruction. Everything else in this packet is reporting.

#	DECISION	WHAT I RECOMMEND, AND WHY
1	Plaintext OTP storage (risk 1)	Option (b): move phone verification into our own hashed <code>otp_challenges</code> table and let Better Auth mint the session afterwards. It is more work than (a) but it is the only option that makes the launch path match the standard we already set, and it removes a whole class of "a dump of one table is a full account takeover" incident. If you prefer (c) for launch speed, say so and I will document the acceptance rather than quietly carrying it.
2	Browser tests in CI (risk 2)	Add a small Playwright job to CI in Phase 2 covering exactly three things: unauthenticated 401, signed-in member 403 on an admin route, and the sign-in journey completing. Not a broad UI suite — three assertions that would catch an authorization regression.
3	Where step-up elevation is required	The mechanism exists and is unattached. I need the list of actions that must demand it — my assumption is: request a withdrawal, change payout details, change the phone number on an account. Confirm or amend before Phase 2 wires it, because retrofitting it later means auditing every call site.
4	The seeded commission rates	18 % course and digital, 15 % service, 12 % physical, all country-agnostic. These are placeholders I chose to make the engine demonstrable. They are configuration and can change at any time — but they will be in the database from the first real sale onward, so confirm them or replace them now.
5	Referral share	The engine takes a referral rate as basis points <i>of the platform's commission</i> . The reconciliation scenario uses 10 %. No referral rate is seeded and nothing charges one yet. Confirm the model — out of the platform's cut, never off the gross — even if the number comes later.
6	Session lifetime	30 days, refreshed daily, revocable in one statement. That is shorter than typical consumer defaults and longer than a banking default. For a marketplace on shared phones I would keep 30 days but require step-up for anything financial — which is decision 3.
7	OTP policy numbers	5 per phone per hour, 20 per IP per hour, 60 s cooldown, 5 attempts. The per-IP limit is the one I would watch: shared connections and carrier NAT are common, and 20 may be too tight in practice. It is a constant in one file and trivially changed once there is real traffic.
8	SMS provider	Nothing is chosen, so nothing sends. The seam is one interface with two methods. This blocks any real user testing, so it is the practical critical path — more than any code in this phase.
9	Proceed to Phase 2	I am not starting it. Section 18 is a proposal, not a plan in motion.

CORRECTION CARRIED FORWARD FROM THE PHASE 0 PACKET

The first issue of the Phase 0 packet stated that CI had never run. **That was false.** CI had already run and passed twice — runs #1 and #2, on commits `87743b6` and `ee04b8f` — before I wrote the sentence. I did not check, and asserted the pessimistic thing as if it were established.

It was corrected in six places in `docs/review/phase-0.html`, with run links, in commit `ee04b8f`. If you reviewed an earlier copy of that PDF, that is the one claim in it that changed. Section 3 of this packet exists in the shape it does because of that mistake.

Against the exit criterion recorded in the Phase 0 packet

That packet proposed this bar for the next phase: *"a real person registers by phone, accepts the terms, creates a store, publishes a paid digital product, and it appears at a public URL — with every step audited and the ledger untouched because nothing has been bought yet."* Your subsequent brief narrowed the phase to the money spine, so the bar splits:

CLAUSE OF THE CRITERION	STATUS	EVIDENCE OR REASON
A real person registers by phone	MET	Section 12 — journey driven end to end. "Real person" still means a console-printed code: no SMS provider exists.
Every step audited	MET	<code>auth.otp.sent</code> , <code>auth.otp.rate_limited</code> , <code>auth.user.provisioned</code> , on an append-only table.
The ledger untouched because nothing was bought	MET	Wallet reads <code>0 X0F</code> derived from entries; reconciliation nets to zero.
Accepts the terms	NOT MET	Consent tables and core functions are tested. No screen presents a document or records agreement.
Creates a store	NOT MET	Out of scope once the phase became the money spine. Phase 2.
Publishes a paid digital product at a public URL	NOT MET	Same. The commission engine is built to receive it.

Three of six clauses are met. I am not calling that a pass — the phase you actually commissioned was the money spine, and against *that* brief every item in section 2 is delivered. But the older bar should not quietly disappear, so it is scored here.

Standing rules from your briefs

RULE	HELD?	WHERE IT IS ENFORCED RATHER THAN PROMISED
No floating-point arithmetic for money	HELD	<code>bigint</code> minor units plus a currency code; <code>money()</code> rejects a non-integer. Property-tested.
Zero-decimal currencies are not special-cased	HELD	The exponent is read from the <code>currencies</code> table. The wallet renders <code>0 XOF</code> .
The ledger is append-only and double-entry	HELD	Four database triggers. A bypass fails loudly.
Never trust frontend validation for financial or permission-sensitive operations	HELD	Every route resolves the actor server-side and calls <code>authorize()</code> . The 403 in section 12 is issued by the server to a client that asked politely.
Nobody approves their own withdrawal	NOT YET REACHABLE	Designed; no withdrawal endpoint exists to enforce it on.
Seller balances are never called deposits or protected funds	HELD	Stated in the API payload itself; no field or label anywhere says otherwise.
Registration is not authorization for regulated activity	HELD	Nothing in this phase claims it is. Risk 5.
Do not invent iPayMoney endpoints, auth, webhooks or responses	HELD	The adapter throws. Three tests stay skipped rather than passing against a fiction.
Do not claim CI is verified until it has run	HELD	Four runs, listed with commits and timestamps in section 3. The Phase 0 breach of this rule is corrected above.
Never say "verified" unless the test was executed	HELD	Section 1 defines the vocabulary; section 3 says where each proof ran.
Stop and ask on decisions affecting money, security, auth or data integrity	HELD	Stopped once mid-phase on the Drizzle/Better Auth question. Stopping again now on plaintext OTP storage — risk 1, decision 1.
Authentication must not become authorization	HELD	Section 6, with the structure drawn out.
Do not blindly replace the users/roles/permissions schema	HELD	One nullable column added. Nothing replaced.
OTP issuance must never remain unlimited	HELD	Section 7, counted atomically in PostgreSQL.
Do not start Phase 2, rebuild the ledger, recreate the blueprint, or discard Phase 0	HELD	None of the four happened.

A PROPOSAL, NOT A PLAN IN MOTION

No Phase 2 work has started, and none will start without your explicit approval.

The money spine can now price a sale, freeze what was charged, and post it without losing an update. What it cannot do is have a sale to price. Phase 2 is the smallest thing that changes that:

1 Whatever section 16 decides

Decisions 1 and 2 in particular. Plaintext OTP storage should be settled before there are real accounts to take over, and the three-assertion browser job should land before more routes exist to regress.

2 Stores and products

A seller creates a store and publishes a paid digital product at a public URL. This is where the commission engine's `subjectType / subjectId` stops being hypothetical.

3 Consent, presented rather than merely stored

The versioned document machinery exists and is tested. Phase 2 shows a document, records the acceptance against its version, and blocks the actions that require it.

4 Checkout against the mock provider, end to end

Order → capture → `freezeSchedule` → post to the ledger → reconcile. Against the mock, because iPayMoney remains undocumented. This is the first time the whole spine carries a real order.

5 The payout path, with the four-eyes rule enforced

A withdrawal request, an approval by someone else, step-up elevation demanded at the moment of approval. This is where decision 3 has to already be answered.

Proposed exit criterion: a buyer purchases a digital product, the seller's entitlement appears in their wallet with the commission frozen and reconciling to zero drift, a withdrawal is requested, and a second person must approve it — with iPayMoney still absent and still not pretended.

THE THING THAT IS NOT CODE

An SMS provider for Niger. Without one, nobody outside this repository can sign in, and none of the above can be tested by a real user on a real handset. It has been the top non-engineering blocker since the Phase 0 packet and it has not moved.

Phase 1 complete · branch `claude/create-app-repository-1cxsih` · head `fcb61db` · GitHub CI run #4 green · **Phase 2 will not begin without explicit approval.**